LINEAR DATA STRUCTURES

UNIT-2: DATA STRUCTURES (ETCCDS202)

Submitted by:

Aryan Solanki
Roll No: 2501730347
School of Engineering & Technology
B.Tech (AI/ML)
Section-A

AMORTIZED ANALYSIS OF DYNAMIC ARRAY .APPEND()

A dynamic array initially has a set capacity. When the array is filled, a new array with a capacity twice as large is created, all the old elements are transferred, and the new element is then added. Doubling the capacity is the main idea that allows efficient average-case performance.

WHY MOST APPENDS ARE O(1)?

Imagine appending n items to a dynamic array that is empty at the beginning. Only a small number of the n operations will require a resize at sizes 1 2 4 8 16 (powers of 2). The total amount of copying required at all resizes combined is: $1 + 2 + 4 + 8 + ... + n = 2n - 1 = O(n)$ Dividing evenly over n append operations, the average cost per append turns out to be $O(n)/n = O(1)$ amortized. Renaming one append if a resize triggers it costs O(n) by itself, but this cost is allocated also to all the previous cheap O(1) appends. Example: Imagine a rechargeable torch. Most of the time it simply works (O(1)). Sometimes you are spending your time recharging it (O(n) resize), but you recharge so rarely that the average time per use is constant.

TIME COMPLEXITY ANALYSIS

1. **Singly Linked List Operations**

Operation	Time Complexity	Reason
insert_at_start()	O(1)	Direct head pointer update
insert_at_end()	O(n)	Must traverse to last node
delete_by_value()	O(n)	Must search linearly for value
traverse()	O(n)	Visits every node once
search()	O(n)	Linear scan in worst case

2. **Doubly Linked List Operations**

Operation	Time Complexity	Reason
insert_after_node()	O(n)	Must find target first
delete_at_position()	O(n)	Must traverse to position
Head or Tail insert	O(1)	Direct pointer update

3. **Stack (LIFO) using Singly Linked List**

Operation	Time Complexity	Reason
push()	O(1)	insert_at_start - head update
pop()	O(1)	Remove head - direct access
peek()	O(1)	Read head.data - no traversal

The key insight for Stack efficiency is to perform both push and pop at the head of the linked list. This avoids traversal entirely, giving O(1) for both operations. If we used the tail instead, pop would require O(n) traversal to reach the second-to-last node.

4. **Queue (FIFO) using Singly Linked List**

Operation	Time Complexity	Reason
enqueue()	O(1)	Tail pointer - direct insert at end
dequeue()	O(1)	Head pointer - direct remove from front
front()	O(1)	Read head.data - no traversal

The Queue maintains both a head and tail pointer. Enqueue happens at the tail (O(1)), and dequeue happens at the head (O(1)). Without the tail pointer, enqueue would require O(n) traversal each time.

BALANCED PARENTHESES CHECKER

The checker uses the Stack ADT built on top of SinglyLinkedList. For each character in the expression:

- If it is an opening bracket ({ [, push it onto the stack.

- If it is a closing bracket, check if the stack top matches the expected opener.

- If the stack is empty at a closing bracket, or top does not match: return False.

- After the full scan, the expression is balanced only if the stack is empty.

Expression	Expected	Result
([])	True	True
(D]	False	False
(((	False	False
empty string	True	True